



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 03

Team No.: 18

Project Title: System Call Demonstration and Monitoring System

Team Members

Roll Number	Student Name	Signature
2520030423	Akhil AD	
2520030424	Revanth Reddy	
2520039623	Advik	

1. Abstract

This project develops a Linux-based System Call Demonstration and Monitoring System to help students understand how user-space applications request services from the Linux kernel. The program demonstrates file operations using `open()`, `read()`, `write()`, and `close()`, and process management using `fork()`, `exec()`, and `wait()/waitpid()`. It checks system-call return values and uses `errno` to report meaningful errors. The program is executed under `strace` so students can observe the system calls generated during file and process operations. The demonstration highlights the boundary between user-space application code and kernel services and shows how common operating-system services are accessed through system calls. The expected outcome is a simple, practical C program that produces clear output and can be traced with `strace`, enabling students to connect source-code operations with the corresponding Linux kernel interactions.

2. Problem Statement

Students often learn system calls theoretically but have limited visibility into what happens when a user program requests services from the Linux kernel. This project addresses that gap by providing a practical C program that performs file and process operations through Linux system calls and then traces those calls using `strace`. It is directly relevant to Operating Systems and Systems Programming because it demonstrates file descriptors, process creation, program execution, process synchronization, return-value checking, and error handling with `errno`.

3. Objectives

- ☐ **To demonstrate file operations** using Linux system calls such as open(), read(), write(), and close().
- ☐ **To demonstrate process management** using fork() for process creation and exec() for executing another program.
- ☐ **To implement process synchronization** using wait() or waitpid() and understand parent-child process execution.
- ☐ **To handle system-call errors** by checking return values and using errno to identify and report errors.
- ☐ **To trace and monitor system calls** using strace and understand the interaction between user-space applications and Linux kernel services.

4. Proposed Methodology

The project will be implemented as a C program in a Linux/Ubuntu environment. First, the program will create or open a file using open(), write data using write(), read the data using read(), and close the file using close(). The program will check the return values of each system call and use errno for error handling.

Next, fork() will be used to create a child process. The child process will use exec() to execute another program, while the parent process will use wait() or waitpid() to synchronize with the child.

Finally, the program will be executed using strace to monitor and display the system calls generated during execution. This will help students understand the transition between user-space application code and Linux kernel services.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
open()	Opens or creates the demonstration file.
read()	Reads data from the file through a file descriptor.
write()	Writes sample data to the file.
close()	Closes the file descriptor after use.
fork()	Creates a child process.
exec()	Replaces the child process image with another program.
wait()/waitpid()	Synchronizes the parent with the child process.
errno / perror()	Reports and handles system-call errors.
strace	Traces and displays system calls generated by the program.
File descriptors	Demonstrates kernel-managed handles used for file I/O.
User space / Kernel space	Shows the boundary between application code and kernel services.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030423	Akhil AD	
2520030424	Revanth Reddy	
2520039623	Advik	

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating system and environment for system-call programming.
C	Implementation of the demonstration program.
GCC	Compiles the C source code.
strace	Monitors and displays system calls and their results.
GNU/Linux terminal	Compiling, executing, and tracing the program.

8. Expected Outcome

- ☐ The system will successfully **create, open, read, write, and close files** using Linux system calls.
- ☐ The system will demonstrate **process creation using fork()**, execution of another program using **exec()**, and synchronization using **wait()** or **waitpid()**.
- ☐ The program will **detect and handle system-call errors** using return-value checking and **errno**.
- ☐ Using **strace**, students will be able to observe and analyze the system calls generated by the program.
- ☐ Students will gain a clear understanding of the **interaction between user-space applications and Linux kernel services**.

9. GitHub Repository

GitHub Repository Link: <https://github.com/revanth-reddy03/System-Call-Demonstration-and-Monitoring-System>

Repository Created: Yes

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: Dr.K.Hema

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____